

Prosper

Platforma badawcza do analizy rynku kryptowalut

Dokumentacja techniczna projektu inżynierskiego

Adrian Budzich
s30076

Repozytorium: <https://github.com/Gusiu/Prosper>
11 czerwca 2026

Spis treści

Spis treści.....	2
1. Wprowadzenie i cel projektu.....	4
1.1 Zastosowane technologie	4
2. Architektura systemu i przepływ danych.....	5
2.1 Główne warstwy architektury	5
2.2 Przepływ danych (end-to-end).....	5
2.3 Struktura katalogów repozytorium	5
3. Backend - szczegółowy opis modułów.....	8
3.1 Konfiguracja (config.py).....	8
3.2 DataManager i Core (core.py)	8
3.3 Pipeline danych - pobieranie i agregacja	8
3.3.1 Binance Client (binance/)	9
3.3.2 Backfill (pipeline/backfill.py)	9
3.3.3 Agregacja (pipeline/aggregate.py)	9
3.4 Inżynieria cech (features/build.py).....	9
3.5 Etykietowanie (labels/)	9
3.5.1 Etykiety kierunkowe i głębokościowe (labels/build.py)	10
3.5.2 Wspólne stałe i helpery (labels/depth.py)	10
3.6 QA - kontrola jakości danych (qa/checks.py).....	10
4. Modele predykcyjne - szczegółowy opis	11
4.1 Baseline (predict/baseline.py).....	11
4.2 ML - HistGradientBoosting (predict/ml.py)	11
4.3 XGBoost (predict/xgboost_model.py).....	11
4.4 GRU - Gated Recurrent Unit (predict/gru.py).....	12
4.5 TFT - Temporal Fusion Transformer (predict/tft.py).....	12
4.6 Format wyjściowy predykcji	12
5. Storage i wersjonowanie predykcji.....	13
5.1 Centralizacja ścieżek (storage/layout.py).....	13
5.2 Zapis predykcji (storage/predictions.py).....	13
5.3 Struktura wersjonowanego katalogu predykcji.....	14
5.4 Struktura katalogu ewaluacji	14
6. Ewaluacja predykcji (eval/predictions.py)	15
6.1 EvaluationSettings	15
6.2 Zaimplementowane metryki.....	15
6.3 Flagi jakości predykcji.....	15
6.4 Przepływ funkcji evaluate_predictions()	16
6.5 Walk-forward ewaluacja (eval/walkforward.py).....	16
7. Backtest strategii tradingowej (eval/backtest.py)	17

7.1 Logika strategii	17
7.2 Obliczane metryki.....	17
7.3 Planner okien akcji (planner/windows.py)	17
8. REST API - FastAPI (api/server.py).....	18
8.1 Endpointy - pełna lista.....	18
8.2 TaskRunner	18
8.3 Middleware i bezpieczeństwo	19
9. CLI - interfejs wiersza poleceń (cli.py).....	20
9.1 Grupy komend.....	20
9.2 Wspólne flagi CLI	20
10. Frontend - interfejs webowy SPA.....	21
10.1 Główne panele UI.....	21
10.2 Zależności frontendowe	21
11. Testy automatyczne.....	22
11.1 Skrypty smoke test.....	22
12. Quickstart - uruchomienie lokalne	23
12.1 Instalacja	23
12.2 Przykładowy pipeline E2E.....	23
12.3 Uruchomienie frontendu.....	23
13. Status implementacji - co jest zrobione, co w toku	24
13.1 W pełni zaimplementowane	24
13.2 Planowane do dopracowania / rozbudowy	24
14. Proponowany plan rozwoju (Roadmap)	26
Etap 1 - Kompletyzacja ewaluacji i backtestingu (2-3 tygodnie)	26
Etap 2 - Rozbudowa API i UI (1-2 tygodnie).....	26
Etap 3 - Optymalizacja modeli (2-3 tygodnie)	26
Etap 4 - DevOps i infrastruktura (1 tydzień)	26
15. Napotkane problemy i rozwiązania	27
16. Podsumowanie	28

1. Wprowadzenie i cel projektu

Prosper jest platformą badawczą klasy research-grade przeznaczoną do analizy rynku kryptowalut opartej na danych SPOT z giełdy Binance. Projekt dostarcza kompletny pipeline eksperymentalny obejmujący pozyskiwanie danych, ich preprocessing, inżynierię cech, etykietowanie, trenowanie modeli uczenia maszynowego, wersjonowanie predykcji, ewaluację jakości predykcji, backtesting strategii tradingowych oraz prezentację wyników przez interfejs webowy i REST API.

Projekt realizowany jest jako praca inżynierska. Jego celem jest demonstracja umiejętności projektowania i implementacji kompletnego systemu analitycznego ML/DS łączącego: backend Python (FastAPI, Polars, PyTorch, XGBoost), interfejs webowy SPA (Vanilla JS), CLI (Typer) oraz zautomatyzowany pipeline danych.

Projekt jest w pełni funkcjonalny w zakresie backfillu danych, preprocessingu, inżynierii cech, etykietowania, predykcji (Baseline, ML, XGBoost, GRU, TFT), ewaluacji, backtestingu, API oraz interfejsu webowego. Najważniejszy moduł w toku dalszego dopracowywania to zaawansowana integracja ewaluacji z backtestem.

1.1 Zastosowane technologie

Warstwa	Technologie
Backend / API	Python 3.12, FastAPI, Uvicorn, Pydantic, Pydantic-Settings
Przetwarzanie danych	Polars (DataFrame), NumPy, PyArrow / Parquet
Machine Learning	scikit-learn (HistGradientBoosting), XGBoost, PyTorch (GRU, TFT)
CLI	Typer, Rich (kolorowe logi i tabele)
Frontend SPA	Vanilla JavaScript, Chart.js, TradingView Lightweight Charts
Przechowywanie danych	Apache Parquet (dane OHLCV), JSONL (predykcje), JSON (metryki)
Testowanie	pytest (testy jednostkowe i integracyjne)
Zarządzanie środowiskiem	Poetry / pip + virtualenv, pyproject.toml
Jakość kodu	Ruff (linting), mypy (typowanie)

2. Architektura systemu i przepływ danych

System zbudowany jest w architekturze warstwowej. Poniższy diagram przedstawia przepływ danych od ich pobrania aż po wyniki backtestingu.

2.1 Główne warstwy architektury

Warstwa	Komponenty	Odpowiedzialność
Warstwa danych (Data Lake)	binance/*, pipeline/*, storage/*	Pobieranie, konwersja, partycjonowanie Parquet
Warstwa cech i etykiet	features/build.py, labels/build.py, labels/depth.py	Inżynieria cech (wskaźniki techniczne), etykiety kierunkowe i głębokości
Warstwa predykcji	predict/baseline.py, ml.py, xgboost_model.py, gru.py, tft.py	Walk-forward training i generowanie predykcji probabilistycznych
Warstwa oceny	eval/predictions.py, eval/backtest.py, eval/walkforward.py	Metryki jakości (Brier, NLL, ECE, Accuracy), backtest trading
Warstwa planowania	planner/windows.py	Generowanie okien akcji (Strong Buy...Strong Sell)
Warstwa API	api/server.py	REST API (FastAPI), TaskRunner, endpoint routing
Warstwa CLI	cli.py	Typy CLI - headless pipeline
Warstwa UI	frontend/app.js, index.html, style.css	SPA: Dashboard, Inventory, Chart, AI Panel, Eval

2.2 Przepływ danych (end-to-end)

- Pobieranie danych: Binance Public Data API → miesięczne pliki ZIP (1-minutowe świece OHLCV)
- Ekstrakcja i konwersja: ZIP → CSV → DataFrame Polars → Parquet (partycjonowanie year/month)
- Agregacja: 1m → 1h, 1d, 1w (osobne partycje Parquet)
- QA: walidacja sortowania, duplikatów, luk, inwariantów OHLC
- Inżynieria cech: wskaźniki techniczne na dziennych danych → features.parquet
- Etykietowanie: forward return → kierunek (long/flat/short) + głębokość (8 binów) → labels.parquet
- Predykcje: walk-forward training → probabilistyczne predykcje P(short/flat/long) per horyzont → predictions.jsonl (wersjonowane)
- Ewaluacja: predykcje vs. rzeczywiste zwroty → Brier score, NLL, ECE, accuracy, score 0-100 → metrics.json + calibration.json
- Backtest: predykcje → sygnały tradingowe (Buy/Sell/Hold) → symulacja portfela → ROI, max drawdown
- Eksport wyników: API REST + UI webowy

2.3 Struktura katalogów repozytorium

Katalog / plik	Zawartość i przeznaczenie
src/prosper/	Główny pakiet Pythona - cały backend, modele i logika aplikacji
src/prosper/api/server.py	FastAPI server - wszystkie endpointy REST, TaskRunner
src/prosper/cli.py	Typer CLI - komendy headless (backfill, aggregate, predict, eval...)
src/prosper/config.py	Pydantic Settings - konfiguracja systemu z obsługą YAML i env vars
src/prosper/core.py	DataManager - inwentarz danych, skanowanie, czyszczenie cache
src/prosper/binance/	Klient HTTP Binance REST, pobieranie ZIP, weryfikacja checksum
src/prosper/pipeline/	backfill.py (pobieranie i zapis Parquet), aggregate.py (resample)
src/prosper/features/build.py	Inżynieria cech: MA, BB, RSI, MACD, ATR, OBV, zmienność intraday
src/prosper/labels/build.py	Etykiety kierunkowe i głębokościowe z forward return
src/prosper/labels/depth.py	Wspólne stałe i helpery: DIRECTION_CLASSES, DEFAULT_HORIZONS, HorizonSpec, parse_depth_bins, assign_depth_bin
src/prosper/predict/baseline.py	Predyktor baseline (empiryczne częstotliwości + wygładzanie Laplace'a)
src/prosper/predict/ml.py	Predyktor HistGradientBoostingClassifier (scikit-learn)
src/prosper/predict/xgboost_model.py	Predyktor XGBoost - eksport feature_importances.json
src/prosper/predict/gru.py	Predyktor GRU (PyTorch) - sekwencyjny model głęboki
src/prosper/predict/tft.py	Predyktor TFT - Temporal Fusion Transformer (PyTorch)
src/prosper/eval/predictions.py	Pełna ewaluacja predykcji: Brier, NLL, ECE, accuracy, score 0-100, kalibracja, rekomendacje
src/prosper/eval/backtest.py	Symulator tradingowy: ROI, max drawdown, liczba transakcji, krzywa kapitału
src/prosper/eval/walkforward.py	Walk-forward ewaluacja po horyzontach
src/prosper/planner/windows.py	Planowanie okien akcji (Strong Buy/Sell, Buy/Sell, Accumulate/Reduce, Hold)
src/prosper/storage/layout.py	Centralizacja ścieżek do plików danych, predykcji i metryk
src/prosper/storage/predictions.py	Zapis predykcji JSONL: monthly i wersjonowane foldery
src/prosper/storage/parquet.py	Helpery Polars: load_parquet, save_parquet, candles_to_dataframe
src/prosper/storage/compact.py	Kompaktowanie plików Parquet
src/prosper/storage/progress.py	Śledzenie postępu pobierania
src/prosper/qa/checks.py	Weryfikacja danych: sortowanie, duplikaty, luki, inwarianty OHLC
src/prosper/utils/	Narzędzia: hash.py (checksum), http.py (retry), time.py (parse_date, generate_month_range)
frontend/	SPA: index.html, app.js (~3400 linii), style.css
tests/	Testy pytest: jednostkowe i integracyjne (16 plików)

Katalog / plik	Zawartość i przeznaczenie
scripts/	Skrypty pomocnicze: smoke_all.py, smoke_check.py, offline_smoke_fixture.py
pyproject.toml	Konfiguracja projektu, zależności, entrypointy
requirements.txt	Lista zależności pip

3. Backend - szczegółowy opis modułów

3.1 Konfiguracja (config.py)

Moduł config.py implementuje konfigurację aplikacji przy użyciu biblioteki Pydantic-Settings. Klasa Settings dziedziczy po BaseSettings i obsługuje automatyczne ładowanie wartości ze zmiennych środowiskowych (prefix PROSPER_) oraz opcjonalnie z pliku YAML.

Konfigurowane parametry:

- data_root - główny katalog danych (domyślnie ./data); nadpisywalny flagą --root w CLI
- raw_data_dir, processed_data_dir, reports_dir, meta_dir - automatycznie wyprowadzane z data_root
- Binance API: binance_public_data_base, binance_api_base, limity pobierania
- QA: qa_gap_tolerance_seconds, qa_flat_threshold_default
- Etykiety: label_flat_threshold_default, label_depth_bins_default
- Parametry baseline modelu: baseline_rolling_window_days, baseline_min_samples
- Parametry planner: planner_short/medium/long_weeks
- Trading simulation: trading_fee_rate, trading_slippage_proxy_rate
- Tryb badawczy (research mode): strict, deterministic, save_metadata, seed

Właściwości (properties) Settings udostępniają ścieżki do podkatalogów danych z zachowaniem spójnej konwencji nazewnictwa: raw_binance_spot_klines_1m_dir, processed_binance_spot_klines_dir, processed_binance_spot_labels_dir, reports_predictions_dir i inne.

Funkcja get_settings() implementuje wzorzec singleton z możliwością tworzenia oddzielnych instancji dla izolowanych środowisk (np. testy smoke z --root ./data_smoke).

3.2 DataManager i Core (core.py)

Klasa DataManager realizuje logikę inwentaryzacji i utrzymania data lake. Jest używana przez API (endpoint /api/data/inventory) i frontend do wyświetlania stanu danych.

Kluczowe metody:

- get_inventory(refresh=False) - zwraca listę słowników opisujących każdy symbol: daty, przedziały czasowe, jakość, rozmiar, obecność cech i etykiet. Wynik jest cache'owany w meta/inventory_cache.json opartym o fingerprint (liczba plików + mtime).
- delete_symbol_data(symbol) - usuwa wszystkie pliki dla danego symbolu w dozwolonych katalogach (raw, processed, reports).
- get_chart_data(symbol, interval, limit, start, end) - pobiera dane OHLCV, cechy i etykiety dla danego zakresu, wykrywa luki czasowe, zwraca JSON gotowy dla Lightweight Charts w frontend.
- _scan_inventory() - równoległe skanowanie z ThreadPoolExecutor (max_workers=10) dla szybkości.
- _interval_gaps() - liczy luki w ciągłości danych (brakujące miesiące/tygodnie) dla każdego interwału.

3.3 Pipeline danych - pobieranie i agregacja

3.3.1 Binance Client (binance/)

Moduł `binance/public_data.py` odpowiada za pobieranie danych historycznych z Binance Public Data. Dla każdego symbolu i miesiąca pobiera plik ZIP ze świecami 1-minutowymi (format: `{SYMBOL}-1m-{YEAR}-{MONTH}.zip`), weryfikuje checksum SHA256 z towarzyszącego pliku `.CHECKSUM` i ekstrahuje CSV.

Moduł `binance/rest.py` implementuje klienta REST do Binance API (m.in. pobieranie listy symboli SPOT z `exchangeInfo`). Obsługuje rate limiting i retry z backoffem.

3.3.2 Backfill (pipeline/backfill.py)

Funkcja `backfill()` realizuje równoległe pobieranie danych historycznych dla podanego symbolu i zakresu miesięcy. Używa `ThreadPoolExecutor` z konfigurowalną liczbą workerów. Dla każdego miesiąca: sprawdza czy dane już istnieją (`skip_existing=True`), pobiera ZIP, ekstrahuje CSV, konwertuje do `DataFrame` Polars i zapisuje jako Parquet w partycjonowanej strukturze katalogów.

3.3.3 Agregacja (pipeline/aggregate.py)

Funkcja `aggregate()` resample'uje dane 1m do wyższych interwałów (1h, 1d, 1w). Dla każdego miesiąca ładuje dane 1m i agreguje OHLCV za pomocą Polars. Wynikowe `DataFrame` są zapisywane do osobnych katalogów Parquet z partycjonowaniem `year/month` (lub `year/week` dla 1w).

3.4 Inżynieria cech (features/build.py)

Funkcja `build_features()` generuje rozbudowany zestaw wskaźników technicznych na podstawie danych kline dla wybranego interwału. Wszystkie cechy są wyznaczane przy użyciu Polars (lazy evaluation, wektorowe operacje).

Generowane cechy (kategorie):

- Logarytmiczne zwroty: `log_return_1b`, `log_return_3b`, `log_return_7b`, `log_return_14b`
- Zmienność rolowana: `rolling_vol_7`, `rolling_vol_14`, `rolling_vol_30` (odchylenie standardowe log-zwrotów)
- Zakres świecy: `candle_range = (high - low) / close`
- Obsunięcia rolowane: `rolling_drawdown_30`, `rolling_drawdown_90`
- Średnie kroczące: `ma_10`, `ma_30`, `ma_diff` (odchylenie MA10 od MA30), `ma_slope`
- Bollinger Bands: `bb_upper`, `bb_mid`, `bb_lower` (± 2 sigma, okno 20)
- RSI: `rsi_14` (Wilder-style EWM, zakres 0-100)
- ATR: `atr_14` (Average True Range, EWM span=14)
- MACD: `macd_hist` (histogram MACD, EMA 12/26/9)
- OBV: skumulowana suma wolumenu ze znakiem (On-Balance Volume)
- Cechy intraday (tylko dla 1d, gdy dostępne dane 1m): `realized_vol_1d` (zrealizowana zmienność), `jump_count` (liczba skoków $> 2\%$), `max_intraday_drawdown`

Wynikowy `DataFrame` jest zapisywany do pliku `features.parquet` w katalogu `data/processed/binance/spot/features/{interval}/symbol={SYMBOL}/`.

3.5 Etykietowanie (labels/)

3.5.1 Etykiety kierunkowe i głębokościowe (labels/build.py)

Funkcja `build_labels()` generuje etykiety uczenia maszynowego na podstawie `forward return` dla wybranego horyzontu (domyślnie 1 dzień). Etykieta kierunkowa to trójklasowy problem klasyfikacji: long ($r > \text{flat_threshold}$), flat ($|r| \leq \text{flat_threshold}$), short ($r < -\text{flat_threshold}$). Domyślny `flat_threshold` = 1%.

Etykiety głębokościowe (`depth_bin`) to 8-klasowy problem regresji binarnej wskazujący, do jakiego przedziału procentowego należy bezwzględny zwrot - stosowane tylko dla long i short. Domyślne biny: 1-2%, 2-3%, 3-5%, 5-8%, 8-13%, 13-21%, 21-34%, 34%+.

3.5.2 Wspólne stałe i helpery (labels/depth.py)

Moduł `depth.py` definiuje wspólne stałe i narzędzia używane przez wszystkie predyktory i moduł ewaluacji:

- `DIRECTION_CLASSES` = ["short", "flat", "long"] - kolejność klas dla wektorów prawdopodobieństwa
- `DEFAULT_HORIZONS` - trójka `HorizonSpec`: short (28 dni), medium (182 dni), long (365 dni)
- `DEPTH_BIN_LABELS` - 8 etykiet binów głębokości
- `direction_from_return(r, flat_threshold)` - mapowanie float $\rightarrow$ string
- `parse_depth_bins(str)` - parsowanie definicji binów z formatu "1-2,2-3,...,34+"
- `assign_depth_bin(value_pct, bins)` - przypisanie wartości do binu

3.6 QA - kontrola jakości danych (qa/checks.py)

Moduł `qa/checks.py` implementuje 4 niezależne sprawdzenia jakości dla każdego pliku Parquet:

- `check_sorted()` - weryfikacja sortowania po kolumnie `open_time`
- `check_duplicates()` - wykrycie duplikujących się znaczników czasu
- `check_time_gaps()` - wykrycie luk (brakujących świec) powyżej tolerancji `qa_gap_tolerance_seconds`
- `check_ohlc_invariants()` - weryfikacja inwariantów OHLC: $\text{high} \geq \max(\text{open}, \text{close})$, $\text{low} \leq \min(\text{open}, \text{close})$, $\text{all} > 0$

Wyniki zapisywane są do pliku JSON w `data/reports/qa/{SYMBOL}/{interval}/{YEAR}-{MONTH}.json`.

4. Modele predykcyjne - szczegółowy opis

Wszystkie modele predykcyjne realizują ten sam wzorzec walk-forward: dla każdego dnia w zadanym zakresie model jest trenowany na danych historycznych z okna poprzednich `train_window_days` dni, a następnie generuje predykcje probabilistyczne $P(\text{short})$, $P(\text{flat})$, $P(\text{long})$ dla trzech horyzontów (`short=28d`, `medium=182d`, `long=365d`).

Predykcje zawierają również rozkłady głębokości (`depth_long_bins`, `depth_short_bins`) - 8 wartości prawdopodobieństwa dla każdego binu głębokości. Wszystkie predykcje są zapisywane do pliku `predictions.jsonl` w wersjonowanym folderze.

4.1 Baseline (predict/baseline.py)

Predyktor Baseline oblicza probabilistyczne przewidywania na podstawie empirycznych częstotliwości z ruchomego okna historycznego (domyślnie 180 dni), stosując wygładzanie Laplace'a dla uniknięcia prawdopodobieństw zerowych.

Dla każdego dnia i horyzontu zlicza ile razy w oknie historycznym wystąpiły klasy `long/flat/short`, dzieli przez łączną liczbę próbek (z Laplace) i zwraca rozkład prawdopodobieństwa. Analogicznie obliczane są rozkłady głębokości dla ruchów `long/short`. Zastosowanie: punkt odniesienia (benchmark) do oceny czy modele ML faktycznie uczą się czegoś istotnego.

4.2 ML - HistGradientBoosting (predict/ml.py)

Predyktor ML używa `HistGradientBoostingClassifier` ze `scikit-learn` jako algorytmu domyślnego. Algorytm ten jest odporny na braki danych, efektywny pamięciowo i szybki na dużych zbiorach danych.

Walk-forward: modele dla trzech horyzontów są retrenowane raz na miesiąc kalendarzowy (na początku nowego miesiąca). Cechy wejściowe to wszystkie kolumny z `features.parquet` z wyjątkiem `open_time`, `close_time`, `close`, `symbol`. Etykiety są obliczane on-the-fly z `forward return`.

Obsługa nierównowagi klas: algorytm automatycznie obsługuje `class_weight`. Dla zapobiegania wycieku danych (`data leakage`): etykiety dla trenowania są obcięte o `h.forward_days` od końca okna treningowego.

Tryb badawczy: przy ustawieniu `--deterministic` lub `--seed` reprodukowalne są dokładne wyniki.

4.3 XGBoost (predict/xgboost_model.py)

Predyktor XGBoost używa `XGBClassifier` z biblioteki `xgboost`. W stosunku do ML (`HistGradientBoosting`) oferuje dodatkową konfigurowalność: `n_estimators`, `max_depth`, `learning_rate`.

Unikalną funkcjonalnością predyktora XGBoost jest eksport `feature_importances.json` do wersjonowanego katalogu predykcji. Plik zawiera ważności cech (`gain`) dla każdego horyzontu, co umożliwia interpretację modelu.

Analogicznie jak ML, retraining odbywa się raz na miesiąc kalendarzowy z tym samym mechanizmem zapobiegania wyciekowi danych.

4.4 GRU - Gated Recurrent Unit (predict/gru.py)

Predyktor GRU implementuje model głębokiego uczenia w oparciu o sekwencyjną architekturę Gated Recurrent Unit (PyTorch). Wejściami są sekwencje o długości `seq_len` (domyślnie 30 dni) cech technicznych.

Architektura modelu: warstwa GRU (`hidden_size=64` domyślnie, 2 warstwy) → warstwa liniowa → wyjście 3 klasy (softmax). Model przetwarza sekwencje przesuwające się (sliding window).

Dataset: klasa `SequenceDataset` implementuje `__getitem__` zwracający parę (sekwencja cech, etykieta kierunkowa). `DataLoader` z `batch_size=32`. Trening przez `epochs=20` epok per okno z kryterium `CrossEntropyLoss` i optymistą Adam.

Checkpointy modeli zapisywane są do katalogu `checkpoints/` z timestampem. Obsługiwane jest `cuda/mps/cpu` (automatyczne wykrycie urządzenia). W trybie deterministycznym: `torch.use_deterministic_algorithms(True)`.

4.5 TFT - Temporal Fusion Transformer (predict/tft.py)

Predyktor TFT implementuje uproszczoną wersję Temporal Fusion Transformer (PyTorch). TFT to zaawansowany model sekwencyjny specjalizujący się w prognozowaniu szeregów czasowych z interpretowalnością - mechanizm self-attention pozwala na identyfikację, które okresy czasowe i cechy mają największy wpływ na predykcje.

Parametry: `seq_len=60` (domyślnie), `max_epochs=10`, `hidden_size=32`. Dłuższe okno czasowe niż GRU pozwala na uchwycenie zależności długoterminowych.

W checkpointach zapisywany jest zarówno stan modelu jak i parametry konfiguracyjne, co umożliwia wznowienie treningu lub inferowanie bez retreningu.

4.6 Format wyjściowy predykcji

Każda predykcja (jeden wiersz JSONL) ma zuniifikowany format:

```
{
  "date": "2024-01-15", "symbol": "BTCUSDT",
  "short": {
    "P_short": 0.21, "P_flat": 0.34, "P_long": 0.45,
    "depth_long_bins": {"1-2": 0.18, "2-3": 0.22, ...},
    "depth_short_bins": {"1-2": 0.20, ...}
  },
  "medium": { ... },
  "long": { ... }
}
```

5. Storage i wersjonowanie predykcji

5.1 Centralizacja ścieżek (storage/layout.py)

Moduł layout.py jest centralnym rejestrem ścieżek dla wszystkich plików danych, predykcji i metryk. Każdy komponent systemu korzysta z funkcji z tego modułu zamiast budować ścieżki bezpośrednio.

Kluczowe funkcje:

Funkcja	Opis
get_parquet_file_path(symbol, interval, year, month)	Ścieżka do pliku Parquet OHLCV: data/processed/binance/spot/klines/{interval}/symbol={S}/year={Y}/month={M}/part.parquet
get_features_parquet_path(symbol, interval)	Ścieżka do cech: data/processed/binance/spot/features/{interval}/symbol={S}/features.parquet
get_labels_parquet_path(symbol, interval)	Ścieżka do etykiet: data/processed/binance/spot/labels/{interval}/symbol={S}/labels.parquet
get_versioned_prediction_dir(symbol, model_type, timestamp, interval)	Katalog wersjonowanej predykcji: data/reports/predictions/{S}/{model}_{interval}_{ts}/
get_versioned_prediction_file(..., filename)	Plik w wersjonowanym katalogu (np. predictions.jsonl, feature_importances.json)
resolve_versioned_prediction_dir(...)	Rozwiązuje ścieżkę z uwzględnieniem starszego formatu nazw folderów
get_evaluation_run_dir(symbol, model_type, timestamp)	Katalog ewaluacji: data/reports/evaluations/{S}/{model}_{ts}/
get_evaluation_metrics_path(...)	Ścieżka do metrics.json w katalogu ewaluacji
parse_versioned_prediction_folder(name)	Parsuje nazwę folderu → (model_type, interval, timestamp)
parse_evaluation_folder(name)	Parsuje nazwę folderu ewaluacji → (model_type, timestamp)
versioned_prediction_folder_name(model_type, timestamp, interval)	Buduje nazwę folderu predykcji
get_prediction_report_path(symbol, year, month)	Ścieżka do miesięcznego pliku JSONL predykcji
get_qa_report_path(symbol, interval, year, month)	Ścieżka do raportu QA
get_recommendation_report_path(symbol, year, month)	Ścieżka do raportu rekomendacji
get_eval_walkforward_summary_path(symbol)	Ścieżka do podsumowania walk-forward

5.2 Zapis predykcji (storage/predictions.py)

Moduł predictions.py implementuje dwa tryby zapisu predykcji:

- `write_predictions_jsonl(predictions, symbol)` - zapis do miesięcznych plików JSONL (`data/reports/predictions/{symbol}/daily/{YEAR}-{MONTH}.jsonl`). Grupuje predykcje według miesiąca YYYY-MM.
- `write_versioned_predictions(predictions, symbol, model_type, interval)` - zapis do wersjonowanego katalogu z timestampem (`data/reports/predictions/{symbol}/{model}_{interval}_{timestamp}/predictions.jsonl`). Zwraca ścieżkę do katalogu.

Kluczowa funkcja `_json_converter(o)` rozwiązuje problem serializacji numpy: konwertuje `numpy.floating` → `float`, `numpy.integer` → `int`, `numpy.ndarray` → `list`. Bez tego funkcja `json.dumps()` rzuciłaby wyjątek dla wartości zwracanych przez modele scikit-learn i XGBoost.

5.3 Struktura wersjonowanego katalogu predykcji

Plik	Zawartość
<code>predictions.jsonl</code>	Wszystkie predykcje dzienne dla danego runu - jeden JSON na wiersz
<code>feature_importances.json</code>	Ważności cech per horyzont (tylko XGBoost) - wynik <code>feature_importances_</code> klasyfikatora
<code>meta.json</code>	Metadane runu (timestamp, seed, parametry) - generowane przy <code>--save-metadata</code>

5.4 Struktura katalogu ewaluacji

Plik	Zawartość
<code>metrics.json</code>	Pełne metryki jakości: <code>schema_version</code> , <code>overall</code> (accuracy, brier, NLL, ECE, <code>model_score</code>), <code>by_horizon</code> , <code>metric_definitions</code> , <code>artifacts</code>
<code>calibration.json</code>	Dane kalibracji: <code>overall ECE</code> , kalibracja per horyzont z binami <code>confidence-accuracy</code>
<code>predictions_quality.jsonl</code>	Jeden JSON na wiersz: data, horyzont, predykcja, rzeczywisty wynik, metryki (brier, NLL, entropy), flagi, jakościowy powód
<code>worst_predictions.csv</code>	Top 100 najgorszych predykcji (posortowane po <code>error_score DESC</code>) - do inspekcji w UI
<code>recommendations.json</code>	Lista predykcji z wyznaczonymi rekomendacjami (Strong Buy...Strong Sell), posortowana po jakości

6. Ewaluacja predykcji (eval/predictions.py)

Moduł eval/predictions.py implementuje kompletny system oceny jakości predykcji probabilistycznych. Jest to jeden z najbardziej rozbudowanych modułów projektu.

6.1 EvaluationSettings

Klasa EvaluationSettings (frozen dataclass) zawiera konfigurowalne progi do ewaluacji:

- flat_threshold - próg flat dla klasyfikacji kierunku (domyślnie 0.01)
- calibration_bins - liczba binów kalibracji (domyślnie 10)
- low_confidence_threshold - próg niskiej pewności (0.45)
- high_entropy_ratio - stosunek entropii do max (0.92)
- confident_wrong_threshold - pewna błędna predykcja (0.65)
- large_return_error_pct - duży błąd głębokości (5%)
- worst_predictions_limit - limit najgorszych predykcji (100)

6.2 Zaimplementowane metryki

Metryka	Opis i interpretacja
Accuracy	Udział predykcji, gdzie przewidziany kierunek zgadza się z rzeczywistym. Miara podstawowa, wrażliwa na nierównowagę klas.
Brier Score (multiclass)	Suma kwadratów różnic między przewidywanymi prawdopodobieństwami a wektorami one-hot. Zakres 0-2. Niższy = lepiej. Mierzy zarówno dokładność jak i kalibrację.
NLL - Negative Log-Likelihood	Ujemny logarytm prawdopodobieństwa przypisanego do rzeczywistej klasy. Penalizuje nadmiernie pewne błędne predykcje. Zakres 0- ∞ .
ECE - Expected Calibration Error	Błąd kalibracji: średnia ważona $ accuracy_bin - confidence_bin $ po binach pewności. Mierzy zgodność pewności z rzeczywistą trafnością. Ideał: ECE=0.
Entropy (Shannon)	Entropia predykcji w jednostkach naturalnych. Wysoka entropia = niski poziom pewności / równomierny rozkład. $\log(3)$ to max dla 3 klas.
KL Divergence (głębokość)	Rozbieżność Kullbacka-Leiblera między rozkładem głębokości one-hot (rzeczywistość) a przewidywanym rozkładem głębokości.
Jensen-Shannon Divergence	Symetryczna wersja KL. $JSD \in [0, \ln 2]$. Odporniejsza na zerowe prawdopodobieństwa.
MAE/RMSE oczekiwanej głębokości	Średni/kwadratowy błąd między oczekiwaną głębokością procentową a bezwzględnym rzeczywistym zwrotem procentowym.
Model Score (0-100)	Złożona ocena jakości: 35% accuracy + 25% Brier + 20% NLL + 10% ECE + 10% RMSE głębokości. Agregowany po horyzontach ważoną średnią.

6.3 Flagi jakości predykcji

Dla każdej predykcji wyznaczane są flagi diagnostyczne:

- low_confidence - pewność poniżej progu (0.45)
- high_entropy - entropia powyżej 92% maksimum
- calibration_mismatch - pewna (>65%) błędna predykcja

- `large_return_error` - błąd głębokości $\geq 5\%$
- `depth_mismatch` - przewidziany bin głębokości $\neq$ rzeczywisty
- `missing_ohlc` - brak odpowiedniej świecy OHLC dla daty predykcji
- `missing_target` - brak świecy docelowej (koniec zakresu danych)

6.4 Przepływ funkcji `evaluate_predictions()`

- Wczytuje predykcje z pliku `predictions.jsonl`
- Ładuje dane OHLCV dla całego zakresu (`load_parquet` wszystkich partycji)
- Ładuje snapshoty cech z `features.parquet` (dla kontekstu diagnostycznego)
- Wykrywa luki w danych OHLCV (`_detect_time_gaps`)
- Dla każdej predykcji i horyzontu: normalizuje prawdopodobieństwa, wyznacza kierunek/pewność/entropię, szuka odpowiedniej świecy docelowej, oblicza wszystkie metryki (`brier`, `NLL`, `depth_kl`, `depth_js`, `error_score`)
- Agreguje metryki per horyzont i ogólnie (`_aggregate_quality`)
- Oblicza ECE z krzywą kalibracji (`expected_calibration_error`)
- Zapisuje: `metrics.json`, `predictions_quality.jsonl`, `calibration.json`, `worst_predictions.csv`, `recommendations.json`

6.5 Walk-forward ewaluacja (`eval/walkforward.py`)

Moduł `walkforward.py` implementuje ewaluację w stylu cross-validation czasowej (`expanding window` lub `rolling window`). Dla zadanego zakresu miesięcy, przesuwając się krokiem `step_months`, oblicza metryki jakości predykcji per horyzont i agreguje wyniki.

Wynik zapisywany jest do `data/reports/eval/{SYMBOL}/walkforward_summary.json`.

7. Backtest strategii tradingowej (eval/backtest.py)

Moduł backtest.py implementuje symulator portfela oparty na predykcjach. Strategia używa predykcji horyzontu short i rekomendacji planera do podejmowania decyzji kupna/sprzedaży.

7.1 Logika strategii

- Ładuje predykcje z miesięcznych plików JSONL dla podanego zakresu dat
- Ładuje ceny zamknięcia (close) z danych Parquet 1d dla każdej daty
- Dla każdego dnia: oblicza $\text{edge} = P_{\text{long}} - P_{\text{short}}$, risk (prawdopodobieństwo dużych ruchów), wywołuje `map_to_recommendation()`
- Buy (Strong Buy / Buy / Accumulate): kupuje wszystko dostępne z potrąceniem `fee_rate`
- Sell (Strong Sell / Sell / Reduce): sprzedaje całą pozycję z potrąceniem `fee_rate`
- Hold: brak działania

7.2 Obliczane metryki

- `roi_pct` - zwrot z inwestycji jako % od `initial_capital`
- `max_drawdown_pct` - maksymalne obsunięcie (peak-to-trough) w %
- `total_trades` - liczba transakcji (Buy + Sell)
- `days_tested` - liczba dni w symulacji
- `final_value` - końcowa wartość portfela
- `chart_data` - lista słowników `{date, action, price, value, recommendation}` do wizualizacji

Obecna implementacja backtestingu jest strategią prostą (all-in / all-out). Planowane usprawnienia: pozycjonowanie frakcyjne, slippage, stop-loss, krótka sprzedaż (patrz sekcja 11 - Roadmap).

7.3 Planner okien akcji (planner/windows.py)

Funkcje `calculate_edge()`, `calculate_risk_metric()` i `map_to_recommendation()` są używane zarówno przez backtest jak i przez ewaluację predykcji.

Rekomendacja	Warunki ($\text{edge} = P_{\text{long}} - P_{\text{short}}$)
Strong Buy	$\text{edge} > 0.30 \text{ AND } \text{risk} < 0.20$
Buy	$\text{edge} > 0.15 \text{ AND } \text{risk} < 0.30$
Accumulate	$\text{edge} > 0.05 \text{ AND } \text{risk} < 0.40$
Hold	$ \text{edge} \leq 0.05$
Reduce	$\text{edge} < -0.05 \text{ AND } \text{risk} < 0.40$
Sell	$\text{edge} < -0.15 \text{ AND } \text{risk} < 0.30$
Strong Sell	$\text{edge} < -0.30 \text{ AND } \text{risk} < 0.20$

Ryzyko jest obliczane jako: $P_{\text{long}} \times \Sigma(P_{\text{duże_biny_long}}) + P_{\text{short}} \times \Sigma(P_{\text{duże_biny_short}})$, gdzie duże biny to 13-21%, 21-34%, 34%+.

8. REST API - FastAPI (api/server.py)

8.1 Endpointy - pełna lista

Metoda	Endpoint	Status	Opis
GET	/api/status	OK	Status serwera
POST	/api/task/run	OK	Uruchom komendę CLI asynchronicznie przez TaskRunner
GET	/api/task/logs	OK	Logi z aktywnego zadania (streaming przez polling, start_idx)
GET	/api/task/status	OK	Status zadania i pasek postępu
GET	/api/data/dates	OK	Bieżące i wczorajsze daty (dla inicjalizacji UI)
GET	/api/data/inventory	OK	Inwentarz danych data lake (z cache'owaniem)
GET	/api/data/klines/{symbol}/{interval}	OK	Dane OHLCV + cechy + etykiety dla wykresu (limit, start, end)
DELETE	/api/data/{symbol}	OK	Usuń wszystkie dane dla symbolu
GET	/api/meta/symbols	OK	Lista symboli SPOT z pliku meta/binance_spot_symbols.json
GET	/api/ai/models	OK	Lista wersjonowanych runów predykcji z metadanymi
POST	/api/ai/train	OK	Uruchom trening modelu (przez TaskRunner)
POST	/api/ai/evaluate	OK	Uruchom ewaluację predykcji (przez TaskRunner)
GET	/api/ai/predictions	OK	Pobierz predykcje (wersjonowane lub miesięczne JSONL)
GET	/api/ai/feature_importances	OK	Pobierz feature_importances.json dla runu XGBoost
DELETE	/api/ai/models/{symbol}/{type}/{ts}	OK	Usuń wersjonowany run predykcji
GET	/api/ai/evaluations	OK	Lista ewaluacji z opcjonalnym filtrowaniem (symbol, model_type)
GET	/api/ai/evaluations/{s}/{type}/{ts}	OK	Szczegóły ewaluacji: metryki, kalibracja, quality rows, recommendations
GET	/api/backtest/{symbol}	OK	Uruchom backtest i zwróć dane wykresu kapitału

8.2 TaskRunner

Klasa TaskRunner zarządza wykonaniem zadań CLI jako procesów potomnych (subprocess.Popen, shell=False). Kluczowe właściwości:

- Jednocześnie może działać tylko jedno zadanie (is_running flaga)
- Logi zbierane linia po linii ze stdout+stderr procesu i przechowywane w liście self.logs
- Postęp śledzony przez parsowanie wzorca [N%] w logach - wyniki mapowane na ogólny procent

- Frontend pobiera logi przez polling GET /api/task/logs?start_idx=N (inkrementalny odczyt)
- Komendy walidowane przez `parse_safe_command_chain()`: tylko `python -m prosper.cli`, bez shell metacharacters, tylko dozwolone subkomendy

8.3 Middleware i bezpieczeństwo

- `CORSMiddleware` z `allow_origins=["*"]` - dostęp z dowolnego origin (środowisko deweloperskie)
- Walidacja symbolu przez regex `[A-Z0-9]{3,30}` przed każdym zapytaniem
- Whitelist komend CLI (`ALLOWED_PROSPER_COMMANDS`) dla `TaskRunner`
- `SYMBOL_RE` regex i `_is_valid_symbol()` chroni przed injection
- `NoCacheStaticFiles` - pliki frontendu serwowane z nagłówkami no-cache (świeże pliki przy dev)

Frontend serwowany jako `StaticFiles` z katalogu `frontend/`. Serwer automatycznie otwiera przeglądarkę po 1.5 sekundy od startu (`threading.Thread` z `webbrowser.open`).

9. CLI - interfejs wiersza poleceń (cli.py)

CLI zaimplementowany jest przy użyciu biblioteki Typer z kolorowymi logami przez Rich. Dostępny jako komenda prosper (po `pip install -e .`) lub `python -m prosper.cli`.

9.1 Grupy komend

Komenda	Opis
<code>prosper backfill</code>	Pobieranie miesięcznych danych 1m z Binance Public Data
<code>prosper aggregate</code>	Resampleowanie 1m → 1h, 1d, 1w
<code>prosper features build</code>	Inżynieria cech → <code>features.parquet</code>
<code>prosper labels build</code>	Generowanie etykiet → <code>labels.parquet</code>
<code>prosper predict baseline</code>	Predykcje Baseline (empiryczne częstotliwości)
<code>prosper predict ml</code>	Predykcje HistGradientBoostingClassifier
<code>prosper predict xgboost</code>	Predykcje XGBoost z feature importances
<code>prosper predict gru</code>	Predykcje GRU (deep learning, PyTorch)
<code>prosper predict tft</code>	Predykcje TFT (Temporal Fusion Transformer)
<code>prosper eval predictions</code>	Ewaluacja wersjonowanego runu predykcji
<code>prosper eval evaluate</code>	Alias dla <code>eval predictions</code>
<code>prosper eval walkforward</code>	Walk-forward ewaluacja po miesiącach
<code>prosper eval backtest</code>	Symulacja tradingowa i wyniki backtestingu
<code>prosper planner windows</code>	Generowanie okien akcji i rekomendacji
<code>prosper qa check</code>	Kontrola jakości danych (sortowanie, duplikaty, luki, OHLC)
<code>prosper symbols list</code>	Pobranie listy symboli SPOT z Binance exchangeInfo
<code>prosper ui</code>	Uruchomienie serwera Web UI (FastAPI + Uvicorn)

9.2 Wspólne flagi CLI

Flaga	Opis
<code>--symbol SYMBOL</code>	Symbol tradingowy (np. BTCUSDT) - wymagany przez większość komend
<code>--start YYYY-MM-DD</code>	Data rozpoczęcia zakresu danych
<code>--end YYYY-MM-DD</code>	Data zakończenia zakresu danych
<code>--root PATH</code>	Katalog root data lake (domyślnie <code>./data</code>). Pozwala na izolację środowisk.
<code>--strict</code>	Tryb rygorystyczny: błąd krytyczny przy lukach w danych
<code>--deterministic</code>	Deterministyczne algorytmy PyTorch (dla reprodukowalności)
<code>--seed INT</code>	Ziarno losowości (numpy, random, torch)
<code>--save-metadata</code>	Zapis <code>meta.json</code> z metadanymi runu obok artefaktów
<code>--interval STR</code>	Interwał czasowy (1m, 1h, 1d, 1w); domyślnie 1d dla <code>predict/eval</code>
<code>--train-window-days</code>	Rozmiar okna treningowego walk-forward (domyślnie 150 lub 365)

10. Frontend - interfejs webowy SPA

Frontend to jednostronicowa aplikacja (SPA) napisana w Vanilla JavaScript bez frameworka. Składa się z trzech plików: index.html (~1100 linii), app.js (~3400 linii), style.css. Aplikacja komunikuje się z backendem przez REST API.

10.1 Główne panele UI

Panel	Funkcjonalność
Data Manager (Inventory)	Macierz dostępności danych: każdy symbol × interwał z kolorowym statusem (complete/warning/incomplete). Przyciski Fix auto-naprawiają braki.
Task Queue / Pipeline Builder	Budowanie i uruchamianie złożonych pipeline'ów (Download→Aggregate→Features→Labels). Logowanie w czasie rzeczywistym przez polling, pasek postępu.
Interactive Chart (Analysis Drawer)	Wysuwany panel z wykresem TradingView Lightweight Charts. Bidirectionalna synchronizacja krzyżyka (main + subcharts RSI, MACD, ATR). Overlaye danych: luki, etykiety, cechy.
AI Panel (Model Training)	Wybór symbolu, modelu, interwału, zakresu dat i parametrów. populateTrainIntervals() wypełnia dostępne okresy. runTrain() wysyła POST /api/ai/train.
Prediction Calendar	Przeglądarka predykcji w formacie kalendarza. openPredictionCalendar() ładuje listę modeli, renderPredictionCalendar() renderuje dzienne predykcje. showPredictionPopover() pokazuje szczegóły per dzień.
Evaluation Panel	Uruchamianie ewaluacji (runPredictionEvaluation), przegląd metryk, kalibracja, lista najgorszych predykcji (renderWorstPredictions), inspekcja pojedynczych predykcji (inspectEvaluationPrediction).

10.2 Zależności frontendowe

- Chart.js - wykresy metrykowe i backtest
- TradingView Lightweight Charts - interaktywny wykres świecowy z synchronizacją krzyżyka
- CSS Custom Properties - theming, responsive design

11. Testy automatyczne

Projekt zawiera 16 plików testowych w katalogu tests/. Testy uruchamiane są przez pytest.

Plik testu	Zakres testowania
test_versioned_predictions.py	Zapis i parsowanie wersjonowanych predykcji (layout, write/read JSONL)
test_predict_interval_integration.py	Integracja: cechy → predykcje ML, XGBoost z izolowanym data lake
test_predict_leakage.py	Weryfikacja braku wycieku danych (data leakage) w walk-forward
test_prediction_evaluation.py	Ewaluacja predykcji: metryki (Brier, NLL, ECE), flagi, format wyjścia
test_api_server.py	Endpointy FastAPI: status, inventory, klines, modele AI, ewaluacje
test_data_manager.py	DataManager: skanowanie inventory, wykrywanie luk, czyszczenie danych
test_aggregate.py	Agregacja kline: 1m → 1h → 1d
test_features_build.py	Budowanie cech: kompletność kolumn, zakres wartości
test_labels.py	Budowanie etykiet: rozkład klas, format wyjścia
test_integration.py	Pełny pipeline E2E: backfill → aggregate → features → labels → predict
test_time_utils.py	Helperzy czasu: parse_date, generate_month_range
test_qa_checks.py	QA checks: sorted/duplicates/gaps/OHLC invariants
test_hash_utils.py	Weryfikacja checksum SHA256
test_planner_windows.py	Planner: calculate_edge, map_to_recommendation
test_cli_exit_codes.py	Kody wyjścia CLI dla scenariuszy sukcesu i błędu
test_eval_walkforward_utils.py	Walk-forward: generowanie przedziałów, format raportów

11.1 Skrypty smoke test

- scripts/smoke_all.py - pełny pipeline smoke test na izolowanym katalogu data_smoke/
- scripts/smoke_check.py - szybka weryfikacja statusu danych i predykcji
- scripts/offline_smoke_fixture.py - test bez połączenia z Binance

12. Quickstart - uruchomienie lokalne

12.1 Instalacja

Opcja A - Poetry (zalecana):

```
cd Prosper
poetry install
poetry run uvicorn src.prosper.api.server:app --reload
```

Opcja B - pip + venv (Windows):

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
pip install -e .          # entrypoint 'prosper'
uvicorn src.prosper.api.server:app --reload
```

12.2 Przykładowy pipeline E2E

```
# 1. Pobierz dane 1m dla BTCUSDT
prosper backfill --symbol BTCUSDT --start 2022-01 --end 2024-12 --workers 4
```

```
# 2. Agreguj do interwałów wyższych
prosper aggregate --symbol BTCUSDT --from 1m --to 1h,1d,1w --start 2022-01 --end 2024-12
```

```
# 3. Zbuduj cechy i etykiety
prosper features build --symbol BTCUSDT --base-interval 1d
prosper labels build --symbol BTCUSDT --base-interval 1d
```

```
# 4. Wygeneruj predykcje XGBoost
prosper predict xgboost --symbol BTCUSDT --start 2023-01-01 --end 2024-12-31
```

```
# 5. Ewaluuj predykcje (podać timestamp z katalogu predictions)
prosper eval predictions --symbol BTCUSDT --model-type xgboost --timestamp 20240101120000
```

```
# 6. Backtest
prosper eval backtest --symbol BTCUSDT --start 2023-01-01 --end 2024-12-31
```

12.3 Uruchomienie frontendu

```
# Otwieranie bezpośrednio w przeglądarce
start frontend\index.html
```

```
# Lub przez serwer statyczny
python -m http.server --directory frontend 8001
```

```
# Lub przez CLI
prosper ui          # uruchamia Uvicorn na porcie 8000 i otwiera przeglądarkę
```

13. Status implementacji - co jest zrobione, co w toku

13.1 W pełni zaimplementowane

Komponent / Funkcjonalność	Uwagi
Pobieranie danych z Binance Public Data	Weryfikacja checksum, retry, parallel workers
Pipeline agregacji 1m → 1h/1d/1w	Polars, partycjonowanie Parquet, idempotentność
QA checks (sorted, duplicates, gaps, OHLC)	JSON raporty per miesiąc
Inżynieria cech (13+ wskaźników technicznych)	MA, BB, RSI, MACD, ATR, OBV, zmienność intraday
Etykiety kierunkowe i głębokościowe	3 klasy × 8 binów, walka z imbalance
Predyktor Baseline (Laplace smoothing)	Punkt odniesienia dla innych modeli
Predyktor ML (HistGradientBoosting)	Walk-forward, anti-leakage
Predyktor XGBoost + feature importances	feature_importances.json per run
Predyktor GRU (PyTorch)	CUDA/MPS/CPU autodetect, checkpointy
Predyktor TFT (PyTorch)	Temporal Fusion Transformer
Ewaluacja predykcji (pełna)	Brier, NLL, ECE, accuracy, model_score, kalibracja, flagi, rekomendacje
Backtest strategii tradingowej	ROI, max drawdown, krzywa kapitału
Walk-forward ewaluacja	Per horyzont, expanding window
Planner okien akcji	7-poziomowa skala rekomendacji
Storage + wersjonowanie predykcji	JSONL, wersjonowane foldery, legacy compat
REST API (FastAPI)	18 endpointów, TaskRunner, walidacja
CLI (Typer)	17 komend, research mode flags
Frontend SPA	Inventory, Chart, AI Panel, Eval, Backtest
Testy automatyczne (pytest)	16 plików testowych, 41+ testów
Smoke tests i skrypty pomocnicze	E2E offline i online

13.2 Planowane do dopracowania / rozbudowy

Priorytet	Komponent	Opis
Wysoki	Backtest	Frakcyjne pozycjonowanie zamiast all-in/all-out, modele slippage, stop-loss
Wysoki	Backtest	Integracja metryk ewaluacji z backtestem (porównanie model_score vs ROI)
Wysoki	API	Endpoint GET /api/ai/metrics/{symbol}/{model}/{ts} dla bezpośredniego dostępu do metryk
Wysoki	Testy	Nowe testy pokrywające evaluate_predictions z kompletem metryk
Średni	Frontend	Dopracowanie wizualizacji kalibracji w panelu ewaluacji

Priorytet	Komponent	Opis
Średni	Frontend	Interaktywna inspekcja predykcji z kontekstem cech (feature snapshot)
Średni	ML	Optymalizacja hyperparametrów (Optuna) dla XGBoost i ML
Średni	Dane	Obsługa danych intraday (1h) jako podstawy dla modeli ML
Niski	DevOps	Konteneryzacja Docker, docker-compose z API + frontend
Niski	DevOps	CI/CD (GitHub Actions): testy, linting, smoke test
Niski	Bezpieczeństwo	Autoryzacja użytkowników API (JWT lub API keys)
Niski	Skalowalność	Celery/RQ zamiast threading dla TaskRunner

14. Proponowany plan rozwoju (Roadmap)

Poniższy harmonogram zakłada kontynuację pracy nad projektem po etapie Erasmus, z priorytetyzacją według wartości merytorycznej dla pracy inżynierskiej.

Etap 1 - Kompletyzacja ewaluacji i backtestingu (2-3 tygodnie)

- Rozbudowa backtestingu o frakcyjne pozycjonowanie: zamiast all-in, każda transakcja angażuje %kapitał proporcjonalny do confidence lub edge modelu
- Implementacja slippage jako parametr (slippage_proxy_rate z Settings)
- Implementacja stop-loss: automatyczne zamknięcie pozycji przy przekroczeniu progu straty
- Dodanie metryki Sharpe Ratio i CAGR do backtestu
- Integracja: porównanie model_score (z ewaluacji) z ROI/Sharpe (z backtestingu) na jednym dashboardzie
- Nowe testy jednostkowe dla pozycjonowania frakcyjnego i stop-loss

Etap 2 - Rozbudowa API i UI (1-2 tygodnie)

- Endpoint GET /api/ai/metrics/{symbol}/{model_type}/{timestamp} - bezpośredni dostęp do metryk bez danych quality rows
- Endpoint GET /api/ai/compare - zestawienie wielu modeli dla tego samego symbolu
- UI: zakładka porównania modeli (tabela metryk model_score, accuracy, brier, nll dla różnych runów)
- UI: poprawa wizualizacji krzywej kalibracji (confidence-accuracy plot)
- UI: eksport danych do CSV/JSON z poziomu frontendu

Etap 3 - Optymalizacja modeli (2-3 tygodnie)

- Integracja Optuna dla hiperoptymalizacji XGBoost (n_estimators, max_depth, learning_rate, colsample_bytree)
- Ensemble: łączenie predykcji Baseline + ML + XGBoost (np. ważona średnia po odwrótości Brier score)
- Obsługa danych intraday (1h) jako dodatkowego źródła cech dla modeli daily (1d)
- Eksperymenty z LSTM jako alternatywą dla GRU

Etap 4 - DevOps i infrastruktura (1 tydzień)

- Dockerfile z multi-stage build: builder (instalacja zależności) + runtime (Alpine)
- docker-compose.yml: serwis API + serwis frontend (nginx)
- GitHub Actions workflow: pytest + ruff + mypy + smoke test przy każdym push
- Pre-commit hooks: ruff format, mypy

Etap 4 jest niskim priorytetem dla pracy inżynierskiej, ale istotnym dla produkcyjnego wdrożenia systemu.

15. Napotkane problemy i rozwiązania

Problem	Rozwiązanie
Numpy scalars nie serializowały się do JSON (TypeError: Type numpy.float32 not serializable)	Implementacja funkcji <code>_json_converter(o)</code> w <code>storage/predictions.py</code> obsługującej <code>numpy.floating</code> , <code>numpy.integer</code> , <code>numpy.ndarray</code> i obiekty z metodą <code>.item()</code>
Konieczność centralizacji ścieżek - różne moduły budowały ścieżki na własną rękę	Moduł <code>storage/layout.py</code> jako centralny rejestr funkcji generujących ścieżki. Wszystkie moduły importują z <code>layout.py</code> .
Wersjonowanie folderów predykcji - legacy vs. nowy format nazw	Funkcja <code>resolve_versioned_prediction_dir()</code> próbuje kilka wariantów nazw dla wstecznej kompatybilności
Wycieki danych (data leakage) w walk-forward	Etykiety dla trenowania są obcinane o <code>h.forward_days</code> od końca okna - <code>safe_end = train_end_idx - h.forward_days</code> . Testy w <code>test_predict_leakage.py</code> weryfikują tę właściwość.
Race condition przy równoległym skanowaniu inventory (ThreadPoolExecutor)	Wyniki zestandaryzowane przez <code>sort</code> przed <code>ThreadPoolExecutor.map()</code> , cache opiera się na fingerprint a nie na stanie <code>thread-local</code>
Pliki binarne DOCX generowane jako artefakty w repozytorium	Usunięcie plików <code>.docx</code> z repozytorium i dodanie do <code>.gitignore</code> . Raporty generowane on-demand.
Duplikaty wierszy po concat wielokrotnych plików Parquet	<code>unique(subset=["open_time"], keep="first")</code> po każdym concat w <code>DataManager</code> i modułach predykcji

16. Podsumowanie

Projekt Prosper stanowi kompletną platformę badawczą do analizy rynku kryptowalut. Zaimplementowany system obejmuje cały łańcuch wartości od pobierania surowych danych giełdowych, przez ich przetwarzanie i inżynierię cech, po trenowanie modeli ML/DL, ewaluację jakości predykcji i symulację strategii tradingowych.

Kluczowe osiągnięcia techniczne projektu:

- Kompletny pipeline E2E: od danych 1-minutowych z Binance do probabilistycznych predykcji z oceną jakości
- 5 implementacji modeli predykcyjnych (Baseline, HistGradientBoosting, XGBoost, GRU, TFT) w zunifikowanym interfejsie walk-forward
- Rozbudowany system ewaluacji z 9 metrykami jakości (Brier, NLL, ECE, accuracy, entropy, KL/JS divergence, MAE/RMSE głębokości, model_score 0-100)
- REST API z 18 endpointami, asynchronicznym TaskRunnerem i bezpieczną walidacją komend
- Interaktywny interfejs webowy z TradingView Lightweight Charts i bidirectionalna synchronizacją
- Research mode: reproducible experiments (--deterministic, --seed, --save-metadata)
- Solidna pokrycie testami automatycznymi (16 plików testowych)

Projekt jest aktywnie rozwijany. Planowane dalsze prace (Roadmap - Sekcja 14) koncentrują się na: frakcyjnym pozycjonowaniu i stop-lossie w backtestingu, optymalizacji hiperparametrów modeli, rozbudowie API o endpoint porównania modeli oraz konteneryzacji.

Dokumentacja odzwierciedla stan repozytorium na branchu maister na dzień 11 czerwca 2026. W przypadku pytań dotyczących implementacji poszczególnych modułów, autor jest dostępny.